Roteiro de estudo: MLP para o PIBIT

Oct 15, 2026 · @Igor Rafael de Carvalho Oliveira

Objetivo e como estudar

Em 4 semanas, de 5 a 6 horas por semana, você sai do zero e consegue: treinar uma regressão e um MLP com os dados públicos do eFlesh, avaliar os dois sem se enganar, escolher qual vai para o ESP32 e explicar a escolha para o seu colega e para a banca.

Você não precisa virar especialista em redes neurais. Para o PIBIT, um MLP é só uma função ajustável que transforma 3 números (Bx, By, Bz) em 1 número (força). O roteiro foca nisso.

Como estudar, com base em evidência. Uma revisão grande de técnicas de estudo (Dunlosky et al., 2013, *Psychological Science in the Public Interest*) classificou duas técnicas como de alta utilidade:

- **Prática de recuperação (testar-se):** ao fim de cada sessão, feche o material e explique em voz alta, ou escreva, o que aprendeu. Cada semana abaixo termina com perguntas para isso.
- **Prática distribuída:** três sessões de 2 horas rendem mais que uma de 6. Reveja rapidamente a semana anterior antes de começar a seguinte.

Reler e grifar foram classificados como de baixa utilidade. Prefira rodar o código e mudar um parâmetro de cada vez: é a prática de recuperação aplicada à engenharia.

Se bater o hiperfoco, use-o nos **desafios extras** de cada semana, e não para pular etapas.

Semana 1 — Regressão: a base de tudo

Um MLP é uma regressão empilhada. Se você entende a regressão, entende 80% do MLP.

Conceitos (em ordem)

1. **Dados de entrada (X) e saída (y).** No seu caso, X = (ΔBx, ΔBy, ΔBz) e y = força em newtons. Cada leitura do sensor é uma linha.
2. **Regressão linear:** força = a·ΔBx + b·ΔBy + c·ΔBz + d. Os números a, b, c, d são os *pesos*; "treinar" é encontrá-los.
3. **Função de custo (MSE):** a média dos erros ao quadrado. O treino procura os pesos que deixam esse número o menor possível.
4. **RMSE:** a raiz do MSE, na mesma unidade da força. É o número que o eFlesh reporta (0,27 N).

5. **Regressão polinomial:** inclui termos como ΔBz^2 e $\Delta Bx \cdot \Delta Bz$. Ainda é "linear nos pesos", mas capta curvas. Foi o que o Barthel (2026) usou.
6. **Por que o campo não é linear com a força:** o campo cai com $1/d^3$ e o TPU não é uma mola perfeita. Por isso a reta erra nas pontas da faixa.

Prática (2–3 sessões)

- [ ] Rodar o código da seção "Código inicial" até a parte de regressão linear.
- [ ] Fazer o gráfico força real × força prevista. Pontos fora da diagonal mostram onde o modelo erra.
- [ ] Trocar a regressão linear pela polinomial de grau 2 e depois de grau 3. Anote o RMSE de cada uma.

Teste-se (sem consultar): o que é um peso? Por que se eleva o erro ao quadrado? Por que a polinomial erra menos que a linear nos seus dados?

Desafio extra: derive à mão a fórmula da regressão linear com uma variável (mínimos quadrados) e confira com o resultado do scikit-learn.

Semana 2 — O que é um MLP

MLP (*multilayer perceptron*, perceptron multicamadas) é uma sequência de camadas. Cada neurônio faz uma regressão linear sobre a camada anterior e passa o resultado por uma função de ativação, que "dobra" a reta. Empilhando camadas, a rede consegue aproximar curvas complicadas sem que você precise escolher o grau do polinômio.

Conceitos (em ordem)

1. **Neurônio:** soma ponderada das entradas + viés, passada por uma ativação.
2. **ReLU:** a ativação mais usada: devolve o valor se for positivo e zero se for negativo. É ela que permite as curvas.
3. **Camadas ocultas:** as do meio. O eFlesh usa 2 camadas de 128 neurônios: "MLP 2×128".
4. **Número de parâmetros:** com 3 entradas, um MLP 2×128 tem $3 \cdot 128 + 128 + 128 \cdot 128 + 128 + 1 = 17.153$ pesos. Uma regressão polinomial de grau 2 tem 10. Mais parâmetros = mais flexibilidade e mais risco.
5. **Treino por gradiente descendente:** ajustar os pesos um pouco de cada vez na direção que reduz o erro. *Backpropagation* é só o jeito eficiente de calcular essa direção para todas as camadas.
6. **Hiperparâmetros:** o que você escolhe antes de treinar: número de camadas e neurônios, taxa de aprendizado (*learning rate*), número de épocas.

7. **Normalização:** colocar entradas e saída numa escala parecida (média 0, desvio 1). Sem isso, o MLP treina mal. No teste com os dados do eFlesh, esquecer de normalizar a saída já atrapalhou.
8. **Overfitting (sobreajuste):** o modelo decora os dados de treino e erra em dados novos. Com 17 mil pesos e poucos ensaios, é o maior risco.

Prática

- [ ] Rodar a parte do MLP no código inicial.
- [ ] Variar o tamanho: (8, 8), (32, 32), (128, 128). Anote RMSE e número de parâmetros numa tabela.
- [ ] Rodar com e sem `StandardScaler` e comparar.

Teste-se: o que a ReLU faz e por que sem ela o MLP viraria uma regressão linear? Por que um modelo maior pode ser pior?

Desafio extra: implementar um MLP de 1 camada só com NumPy (sem scikit-learn), treinando com gradiente descendente. É o melhor jeito de entender *backpropagation*.